Pretraživanje 1

Predavanje 2: kako nešto pronaći, i to brzo

dr.sc.tech. Kristijan Poje

Zašto algoritam i strukturu podataka učimo zajedno?

- Struktura podataka jest *obećanje* o načinu na koji su podaci raspoređeni.
- Algoritam je ono što to obećanje *održava* dok se podaci mijenjaju.
- Jedno bez drugoga nema mnogo smisla: sortirano polje obećava "sortiranost", a binarno pretraživanje radi samo *zato što* to obećanje vrijedi.

Zašto se time bavimo: cijena neorganiziranosti

- Primjer iz stvarnog života: vaši ključevi. Bacite ih bilo kamo → svaka pretraga provjerava sve.
- Svaki ih put stavite na istu kukicu → njihovo pronalaženje prestaje uopće biti pretraživanje.
- Trud organiziranja ulaže se jednom, a vraća se pri svakoj sljedećoj pretrazi.

Što predavanje obuhvaća: ljestve, a ne popis

Šest struktura, od kojih svaka popravlja ono što nije valjalo u prethodnoj:

1. Sekvencijsko pretraživanje — provjerava sve (polazna osnova)
2. Sortirano polje + binarno pretraživanje — uređenost omogućuje preskakanje polovice ključeva, ali je mijenjanje poretka skupo
3. Binarno stablo pretraživanja — jeftino se mijenja, ali može postati nesimetrično
4. Crveno-crno stablo — isto stablo, uz pravilo koje sprječava da se previše nagne
5. B-stablo — napravljeno za disk ili mrežu, gdje *dohvat* košta više od *usporedbe*
6. LSM-stablo — napravljeno za radna opterećenja u kojima je *pisanje* češće od *čitanja*

Jedno pitanje na koje odgovara svaka struktura

Uređeni rječnik (dinamički skup) odgovara na istih šest pitanja, bez obzira na to kako je izgrađen u pozadini:

- `search(x)` — nalazi li se x ovdje?
- `insert(x)` / `delete(x)` — dodaj ili ukloni
- `min()` / `max()` — najmanji / najveći
- `floor(x)` / `ceiling(x)` — najbliži ključ manji od ili jednak x / veći od ili jednak x, čak i ako x nije prisutan
- `rank(x)` / `select(k)` — koliko je ključeva manje od x? koji se ključ nalazi na položaju k?

Odjeljak 1 — Sekvencijsko pretraživanje

Problem: kamo sam stavio ključeve?

- Ključevi mogu biti bilo gdje u stanu. Provjerite kauč — ništa. Provjerite radnu plohu — ništa.
- Svaka neuspješna provjera ne govori vam *ništa* o tome gdje tražiti sljedeće.
- To je nasumično provjeravanje / provjeravanje jednog mjesta za drugim.

Formalno o sekvencijskom pretraživanju

- Pogledajte jedno mjesto. Nema ničega? Prijeđite na sljedeće. Niste ništa mudriji nego na početku.
- Najgori slučaj: provjerite baš svako mjesto prije nego što pronađete ključ ili se uvjerite da ga nema.
- Za N ključeva neuspješna pretraga košta točno N provjera, a nijedna pretraga nikad ne treba više.

Trošak: O(n), i nikad se ne poboljšava

- Trošak pretraživanja: do N provjera. Udvostručite zbirku i udvostručili ste posao — jedan prema jedan.
- Trošak *izgradnje* zbirke od N ključeva, uz provjeru duplikata pri svakom novom ključu: ukupno približno $N^2/2$ usporedbi.
- Sekvencijsko je pretraživanje pošten rad, ali u hodu ne postaje nimalo pametnije.

Što sekvencijsko pretraživanje donosi te koliko košta

Donosi	Košta
Ne treba ništa izgraditi ni održavati	Nijedno pitanje nikad nije jeftino — svaki put O(n)

Odjeljak 2 — Sortirano polje i binarno pretraživanje

Namjerno posložite ključeve

- Analogija s privjeskom za ključeve: rasporedite ključeve po redu, od manjeg prema većem.
- "Provjeri sve" pretvara se u "provjeri u blizini" — uređenost vam omogućuje da otprilike predvidite gdje bi ključ trebao biti.
- Jedna promjena: privjesak nema krajeva, pa nema ni sredine u koju bismo pogledali. Skinite ključeve s privjeska i položite ih u **red**, od prvog do posljednjeg.
- Taj uređeni red programeri nazivaju **sortiranim poljem**.

Binarno pretraživanje: prepolovljavanje umjesto provjeravanja

- Pogledajte srednji ključ. Cilj je manji → odbacite cijelu gornju polovicu. Veći je → odbacite donju polovicu.
- Ponavljajte na polovici koja je preostala.
- **Invarijanta** koja to čini sigurnim: ako je ključ uopće prisutan, uvijek se nalazi negdje unutar dijela koji se još razmatra.



Razrađeni primjer

Sortirani red od 15 ključeva: 2, 5, 8, 11, 14, 17, 20, 23, 26, 29, 32, 35, 38, 41, 44. Pronađite 38.

- Pogled 1: srednji je 23 → 38 je veći → odbacite lijevu polovicu (nestalo je 7 ključeva).
- Pogled 2: srednji u preostalom dijelu je 35 → 38 je i dalje veći → odbacite još.
- Pogled 3: srednji je 41 → 38 je manji → odbacite 41 i sve iza njega.
- Pogled 4: preostao je jedan ključ, 38. Pronađen.



Trošak: $O(\log_2 n)$

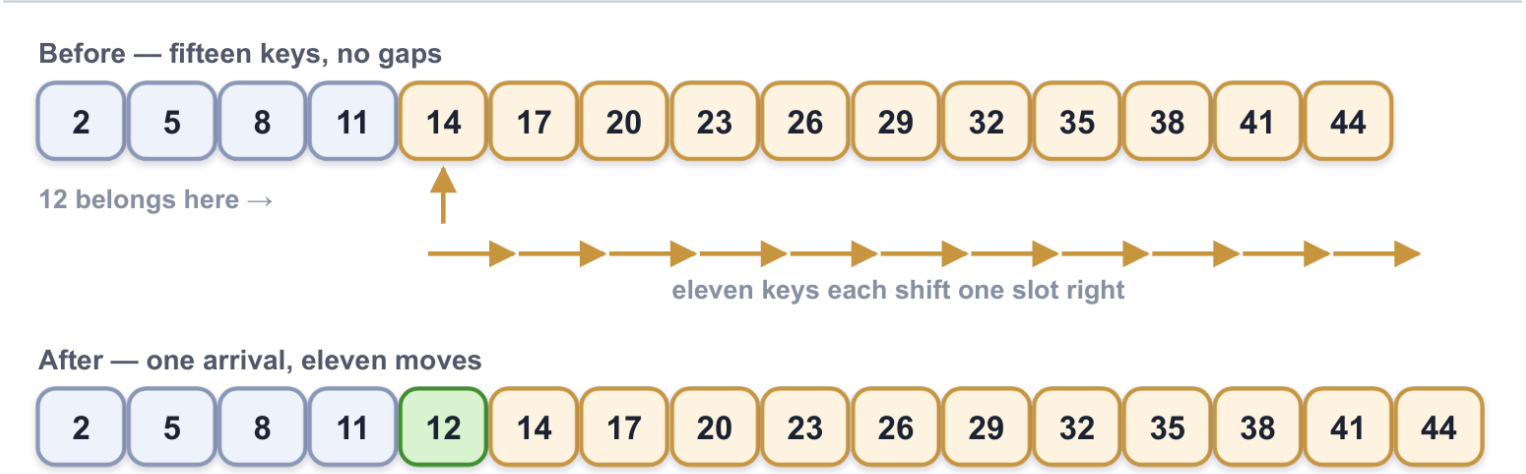
- Udvostručivanje duljine reda dodaje približno **još jednu provjeru**, a ne dvostruko više provjera.
- Skraćeni zapis: $\lg n$ (logaritam po bazi 2).
- Skica dokaza: u najgorem slučaju $C(N) \leq C(\lfloor N/2 \rfloor) + 1$, što razmotavanjem daje $C(N) \leq \lceil \lg(N+1) \rceil$.

Zašto je baza logaritma važna

- Odbacivanje *udjela* u svakom koraku (primjerice polovice) → i dalje O(log n), samo s drugom bazom.
- Odbacivanje *fiksnog broja* u svakom koraku (kao u sekvencijskom pretraživanju, koje pri svakom pogledu odbacuje točno jedan ključ) → O(n), koliko god taj fiksni broj bio velik.
- Stvarna usporedba za pola milijuna računa: binarno pretraživanje u najgorem slučaju treba najviše 19 pogleda. Sekvencijsko je pretraživanje trebalo do pola milijuna.

Cijena mijenjanja sortiranog polja

- Umetnite ključ 12 u sortirani red: pronalaženje njegova mjesta jeftino je (binarno pretraživanje, ~4 pogleda).
- Smjestiti ga ondje nije jeftino: svaki se ključ od tog mjesta nadalje pomiče za jedno mjesto kako bi oslobodio prostor. Jedanaest se ključeva pomakne da bi jedan stigao.
- U velikom mjerilu: pola milijuna sortiranih ključeva → 19 pogleda za pronalaženje mjesta, a zatim se **u prosjeku pomakne četvrt milijuna ključeva**.



Jednodimenzionalna zagušenost

- Pravi uzrok: rang ključa *jest* njegov položaj. Ničiji se rang ne može promijeniti bez promjene adrese.
- Red (polje) ima samo jedan smjer u kojem može rasti, a jedini je slobodan prostor na samom kraju.
- Naziv ovog neuspjeha: **jednodimenzionalna zagušenost**.

Što sortirano polje i binarno pretraživanje donose te koliko koštaju

Donosi	Košta
O(log n) pretraživanje	O(n) umetanje — promjene su bolne

Odjeljak 3 — Binarno stablo pretraživanja

Razgranati raspored umjesto reda

- Umjesto reda zamislite kukice: svaka kukica (**čvor**) drži ključ, a ispod nje ima mjesta za još dvije — jednu dolje lijevo i jednu dolje desno.
- Prva kukica, ona ispod koje visi sve ostalo, jest **korijen**.
- Red ima jedan smjer i ponestane mu prostora. Grananje ima drugi smjer i prostora mu nikad ne ponestane.

Terminologija (rabite je od prvog dana — svi ostali to čine)

- **čvor** — ključ i najviše dvije grane; **korijen** — najviši čvor.
- **djeca** — najviše dva čvora koja vise izravno ispod nekog čvora; **roditelj** — čvor iznad njih.
- **braća** — dvoje djece istog roditelja. **list** — čvor ispod kojeg ništa ne visi.
- **podstablo** — čvor i sve što visi ispod njega.

Jedno pravilo koje ga čini pretraživim

`lijevo < čvor < desno`

- Sve u lijevom podstablu čvora je manje. Sve u desnom podstablu je veće.
- Pravilo vrijedi za *cijele strane*, ne samo za dvoje djece: 60 kao desno dijete od 25, koji je lijevo dijete od 50, krši pravilo iako svaki par roditelj–dijete izgleda ispravno.
- Pravilo ne govori ništa o ključu *jednakom* onome u čvoru. Zadani izbor knjige: bez duplikata — `insert` vraća `False` i ništa ne dodaje. Ako duplikati trebaju: polje s brojem pojavljivanja ili popis uz svaki ključ (vidi okvir u §2.4.1 udžbenika).

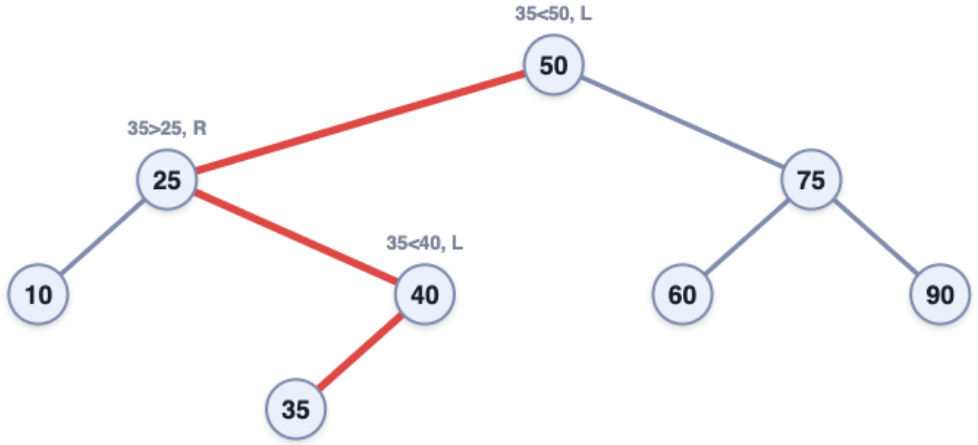
Umetanje: spuštaj se, objesi na prvo prazno mjesto

```
def insert(self, key) -> bool:
    if self.root is None:
        self.root = _Node(key)
        return True
    node = self.root
    while True:
        if key == node.key:
            return False
        elif key < node.key:
            if node.left is None:
                node.left = _Node(key)
                return True
            node = node.left
        else:
            if node.right is None:
                node.right = _Node(key)
                return True
            node = node.right
```

Razrađeni primjer: izgradnja i pretraživanje

Tim redom umetnite 50, 25, 75, 10, 40, 60, 90, 35. Zatim potražite 35:

- Na 50: 35 je manji → idi lijevo, do 25.
- Na 25: 35 je veći → idi desno, do 40.
- Na 40: 35 je manji → idi lijevo, do 35.
- Na 35: to je traženi ključ. Pronađen u četvrtoj usporedbi (tri su usporedbe odabrale smjer, četvrta je pogodila), a da nikad nismo pogledali 75, 60, 90 ni 10.

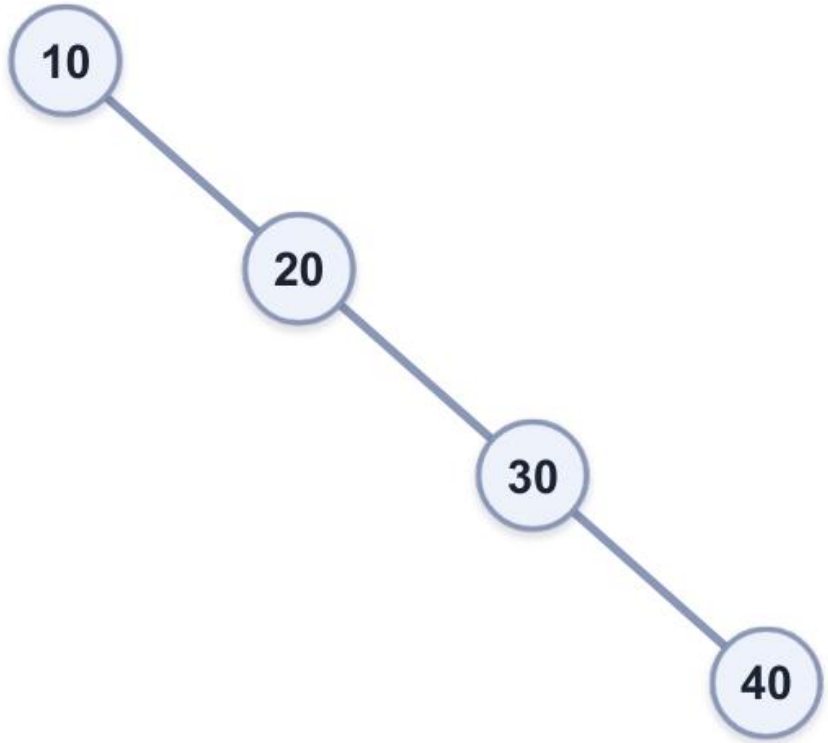


Dodatne operacije, gotovo besplatno

- `min` / `max`: idite do kraja ulijevo ili do kraja udesno.
- `floor(x)` / `ceiling(x)`: isti hod, ali čuvajte najboljeg dosad viđenog kandidata (cilj se možda uopće ne nalazi u stablu).
- `rank` / `select`: potrebno je jedno dodatno polje u svakom čvoru — veličina podstabla — koje se ažurira pri svakom umetanju i brisanju.

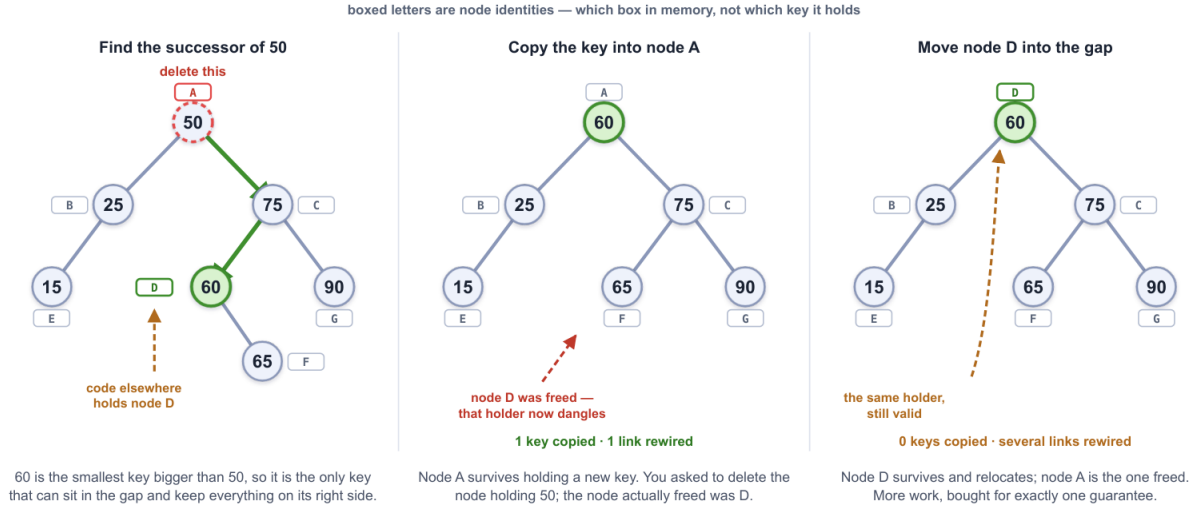
Kvaka: ništa ga ne sprječava da naraste nesimetrično

- U prazno stablo umetnite 10, 20, 30, 40 — već sortiranim redom.
- Svaki novi ključ dosad je najveći → uvijek ide desno. Stablo se nikad ne grana.
- Rezultat: jedan dugačak niz niz desnu stranu — ponovno red, samo odjeven kao stablo. Pretraživanja sada koštaju koliko i pretraživanja reda: O(n).



Brisanje: tri slučaja

- Nema djece — otkvačite ga, gotovo.
- Jedno dijete — dijete se pomiče gore na njegovo mjesto.
- **Dvoje djece** — stvarni slučaj: pronađite **sljedbenika** (najmanji ključ veći od onoga koji odlazi: idite jednom desno, a zatim što dalje lijevo). Smjestite ga u prazninu.



Što obično BST stablo donosi te koliko košta

Donosi	Košta
Jeftino umetanje — ništa se ne pomiče	Visina nije ograničena — može degradirati do O(n)

Odjeljak 4 — Lijevo nagnuto crveno-crno stablo (LLRB)

Motivacija: jeftino ograničiti visinu

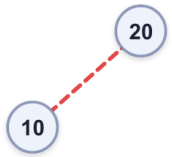
- Trošak običnog BST-a jest O(h) — prihvatljivo ako h ostaje blizu lg n. Nije prihvatljivo ako ključevi stižu sortirani.
- Cilj: pravilo koje sprječava da se stablo previše nagne, provjerava se i popravlja pri svakom umetanju, bez ponovne izgradnje cijelog stabla.

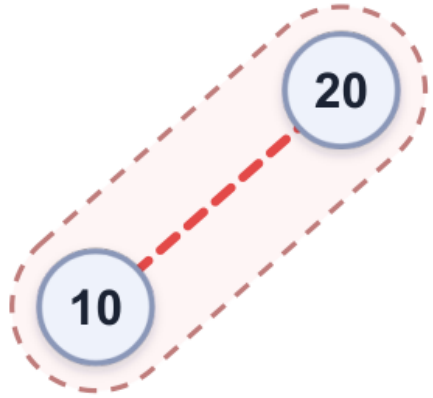
Četiri moguća značenja riječi "uravnoteženo"

- **Savršeno uravnoteženo** — svaki je put iste duljine. Najjači uvjet, ali jedno umetanje može zahtijevati potpunu ponovnu izgradnju.
- **Visinski uravnoteženo u svakom čvoru (AVL)** — dvije strane svakog čvora razlikuju se za najviše 1. Čvrsta granica (~1,44 lg n), ali za održavanje treba mnogo rotacija.
- **Labavo uravnoteženo (izbor ovog predavanja)** — nijedan put nije više nego približno dvostruko dulji od bilo kojeg drugog: O(log n), jeftinije održavanje.
- **Težinski uravnoteženo** — ista ideja, ali broji ključeve umjesto koraka.

Ideja: bojite poveznice, a ne čvorove

- Svakoj poveznici dodijelite jedan dodatni bit: **crveni** ili **crni**.
- Crvena poveznica zapravo nije poveznica — dva čvora koja spaja jedan su čvor koji slučajno sadrži dva ključa.
- Crne su poveznice pravi koraci niz stablo. Crvene su poveznice spojevi unutar (konceptualno šireg) čvora.





one node, two keys

Pet pravila valjanog stabla

1. Ključevi su poredani, kao u svakom stablu pretraživanja.
2. Korijen je crn.
3. Crvena poveznica uvijek se naginje **ulijevo**, nikad udesno.
4. Nijedan čvor nikad ne dodiruju dvije crvene poveznice (ni po jedna sa svake strane ni dvije u lancu ispod njega).
5. Svaki put od korijena do praznog mjesta prelazi isti broj crnih poveznica.

Posljedica: pravila 4 i 5 ograničavaju visinu. Crvene se poveznice ne mogu nizati, pa je najviše svaka druga poveznica na putu crvena; crna je "okosnica" jednaka na svakom putu. Zato najdulji put nikad nije više od približno dvostruko dulji od najkraćeg — stablo je, brojimo li samo crne poveznice, savršeno uravnoteženo, samo tako ne izgleda.

Tri poteza za popravak

```
def rotate_left(h):
    x = h.right; h.right = x.left; x.left = h
    x.red, h.red = h.red, True
    return x

def rotate_right(h):
    x = h.left; h.left = x.right; x.right = h
    x.red, h.red = h.red, True
    return x

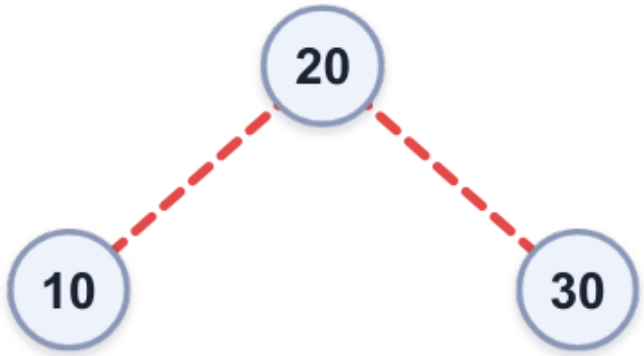
def flip_colors(h):
    h.red = not h.red
    h.left.red = not h.left.red
    h.right.red = not h.right.red
```

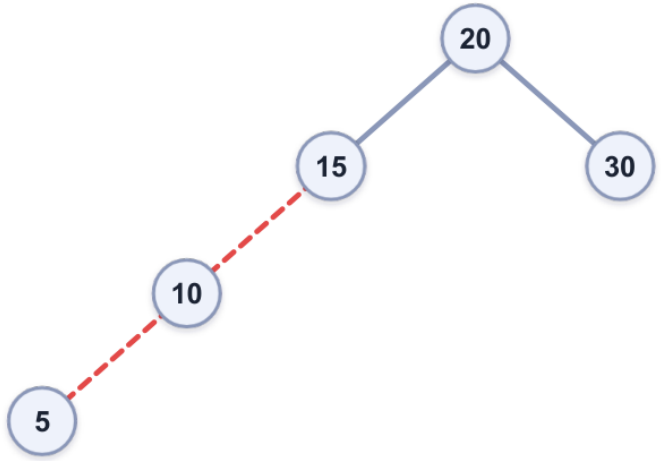
`fix_up`: izvršite sva tri poteza na povratku prema gore

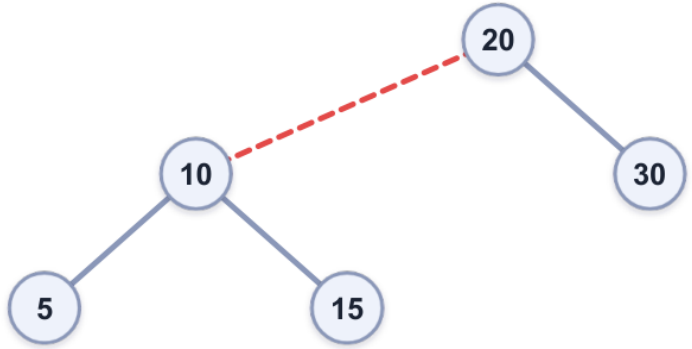
```
def fix_up(h):
    if is_red(h.right) and not is_red(h.left):
        h = rotate_left(h)
    if is_red(h.left) and is_red(h.left.left):
        h = rotate_right(h)
    if is_red(h.left) and is_red(h.right):
        flip_colors(h)
    return h
```

Razrađeni primjer: objesi 10, 20, 30, 15, 5

Objesi	Popravak koji se aktivira	Stablo nakon toga
10	nijedan	samo 10
20	rotacija ulijevo na 10	20, s 10 lijevo na crvenoj poveznici
30	zamjena boja na 20	20, s 10 i 30, oba crna
15	rotacija ulijevo na 10	20, s crnim 15 lijevo i crvenim 10 ispod njega
5	rotacija udesno na 15, zatim zamjena boja na 10	20, s crvenim 10 koji nosi crne 5 i 15 te crnim 30

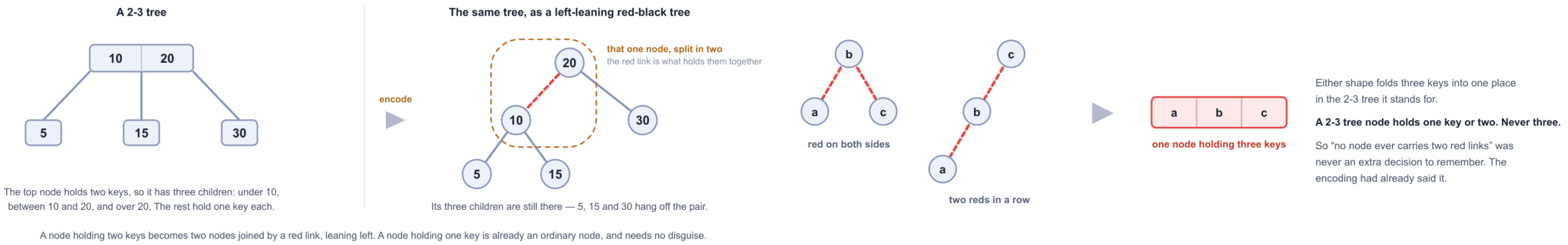






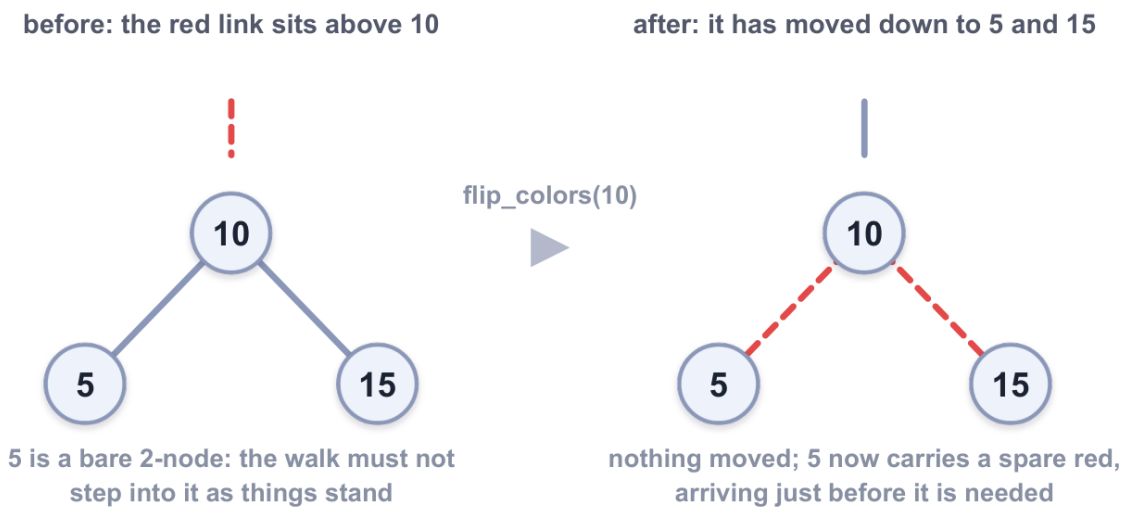
Odakle to dolazi: 2-3 stablo

- **2-3 stablo**: čvorovi sadrže jedan ključ (2-čvor) ili dva ključa (3-čvor), nikad više.
- 3-čvor se kodira kao dva obična čvora spojena crvenom poveznicom — rastavljena samo zato što običan binarni čvor ne može odjednom sadržavati dva ključa.
- Svako pravilo s prethodna četiri slajda izravna je posljedica očuvanja tog prijevoda.



Brisanje (ukratko): pošaljite zalihu dolje prije nego što vam zatreba

- Invarijanta pri spuštanju: čvor u koji ćemo upravo zakoračiti nikad nije goli 2-čvor (ili je sam crven ili ima crvenu poveznicu slijeva).
- `move_red_left` / `move_red_right`: prije spuštanja posudite rezervnu crvenu poveznicu od susjednog čvora kako se hod nikad ne bi zaglavio.
- Ista tri popravka (`fix_up`) pospremaju stanje na povratku.



Što crveno-crno stablo donosi te koliko košta

Donosi	Košta
Zajamčena visina O(log n), bez obzira na redoslijed pristizanja	Malo administriranja (bit boje + provjera popravka) pri svakom umetanju, zauvijek

Odjeljak 5 — B-stablo

Motivacija: dohvat košta više od usporedbe

- Sve su dosadašnje varijacije pretpostavljale da pogled na bilo koji ključ košta približno jednako kao pogled na bilo koji drugi (**model RAM-a**).
- Stvarna memorija daje cijeli blok odjednom — diskovni sektor, stranicu — tražili vi sve to ili ne. Dohvat bloka skup je korak; pregledavanje onoga što je u njemu gotovo je besplatno.
- Stablo s jednim ključem po čvoru uzalud troši gotovo cijeli blok koji dohvati pri svakom koraku.

Popravak: neka čvor drži naručaj ključeva

- Čvor B-stabla sadrži više ključeva odjednom (sortiranih unutar čvora) i ima jedno dijete više nego ključeva.
- Jedan skupi prijenos bloka sada donosi mnogo jeftinih usporedbi unutar čvora, umjesto da plaćamo novi prijenos bloka pri svakom praćenju pokazivača.
- Ništa se drugo u stablu ne mijenja — i dalje se grana, uspoređuje i spušta jednim putem.

Dobitak u broju dohvata

Pohranjeni ključevi	Dohvati, 1.000 ključeva/čvor	Dohvati, 1 ključ/čvor
500.000	3	19
50.000.000	3	26
62.500.000.000	4	36

Pravila

1. Ključevi su poredani strogo uzlaznim redoslijedom, kako unutar svakog čvora, tako i kroz čvorove.

2. Čvor koji nije list ima jedno dijete više nego što ima ključeva.

3. Nijedan čvor nije nikada prepun; svaki čvor osim korijena popunjen je barem do polovice (uz zaokruživanje naniže) i nikada nije prazan.

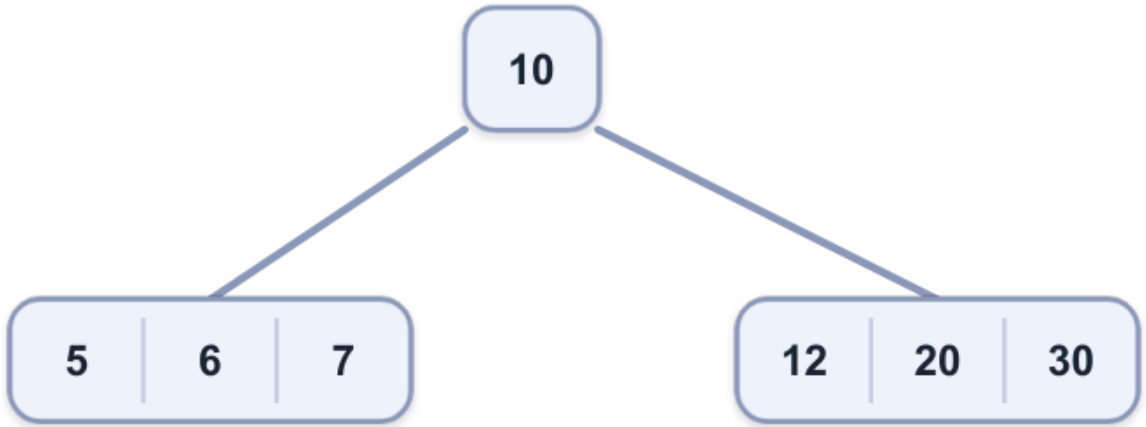
4. Korijen je ili list ili ima barem dvoje djece.

5. Svi se listovi nalaze na istoj dubini.

Razrađeni primjer: izgradnja i pretraživanje

Čvor drži najviše tri ključa (i najviše četvero djece). Umetnite 10, 20, 5, 6, 12, 30, 7, jedan po jedan. Svi osim jednoga samo kliznu u čvor u koji stignu; iznimka je 6 — tada korijen već drži 5, 10, 20 i pun je, pa se najprije cijepa. Nakon svih sedam: korijen sadrži 10; lijevi čvor sadrži 5, 6, 7; desni čvor sadrži 12, 20, 30.

- Potražite 20: u korijenu, 20 > 10 → idite desno. Dohvatite taj čvor (drugi prijenos bloka). Pregledajte njegove ključeve: 12, zatim 20 — podudaranje.
- Dohvaćena su dva čvora, a uspoređena tri ključa (jedan u korijenu, dva ispod). Uravnoteženo stablo s jednim ključem po čvoru dohvatilo bi za istih sedam ključeva najviše tri čvora, ovo najviše dva — s rastom broja ključeva ta razlika postaje tablica sa Slajda 39.



Umetanje: rascijepite prije prelijevanja

- Umetnite 17 u čvor koji je već pun (sadrži najveći dopušteni broj ključeva). Nema mjesta da ga jednostavno umetnete.
- **Najprije rascijepite, prije daljnjeg spuštanja:** uzmite srednji ključ punog čvora i potisnite ga *gore* u roditelja. Dvije polovice s obje strane postaju dva zasebna čvora.
- Tek nakon dovršetka cijepanja broj 17 zaista se smješta — u polovicu kojoj pripada (uz 12).
- Svaki pun čvor na putu dolje cijepa se *prije* ulaska u njega, pa roditelj uvijek ima mjesta za srednji ključ — cijepanje se nikad ne mora vraćati prema gore.



Kôd: cijepanje djeteta

```
def _split_child(self, parent, i):
    t = self.t
    y = parent.children[i]
    z = _BTNode(leaf=y.leaf)
    mid = y.keys[t - 1]
    z.keys = y.keys[t:]
    y.keys = y.keys[: t - 1]
    if not y.leaf:
        z.children = y.children[t:]
        y.children = y.children[:t]
    parent.children.insert(i + 1, z)
    parent.keys.insert(i, mid)
```

Koliko B-stablo može narasti u visinu?

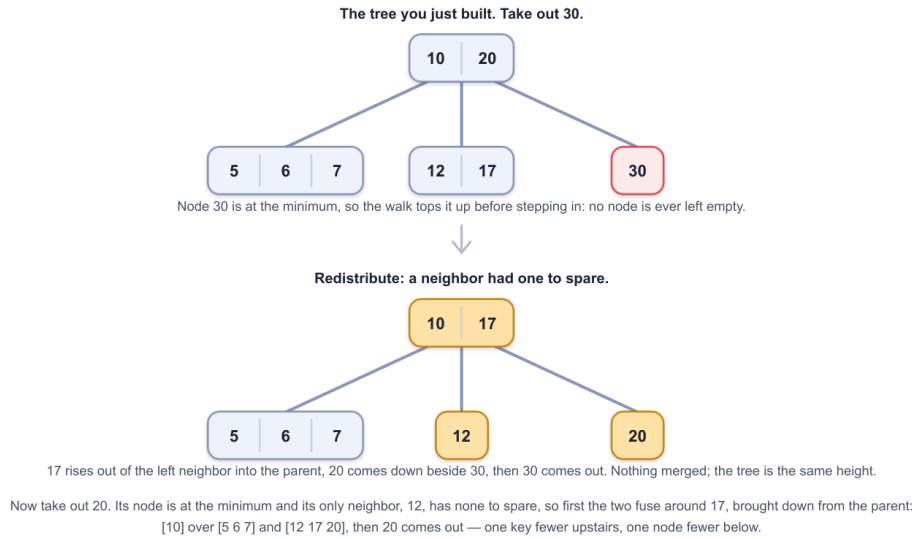
- Visina je najviše log_t((n+1)/2) — spuštanje za jednu razinu množi broj čvorova s najmanje t (svaki čvor osim korijena ima najmanje t − 1 ključeva i t djece).
- Konkretan primjer: blokovi od 16 KB, ~680 ključeva po čvoru → t ≈ 340. **Bilijun ključeva → samo 5 razina**, uvijek, čak i u najgorem slučaju.
- Istih bilijun ključeva u *bilo kojem* binarnom stablu: najmanje ~40 razina (crveno-crno stablo, u najboljem slučaju ~lg n ≈ 40, u najgorem do dvostruko više). Svaki je od tih koraka zaseban dohvat s diska.

Poboljšanje: B*-stablo

- Čvorovi običnog B-stabla pri nasumičnim dolascima u prosjeku se ustale na približno 69 % popunjenosti — otprilike trećina prostora većinu je vremena prazna.
- **B*-stablo:** prije cijepanja punog čvora najprije pokušajte bočno premjestiti ključeve u susjedni čvor koji još ima mjesta. Cijepajte tek kad su i susjedi puni — a tada podijelite dva puna čvora na *tri*, ne na dva.
- Rezultat: prosječna popunjenost raste s ~69 % na ~81 %. Puniji čvorovi, manje njih, uz cijenu provjere susjeda prije svakog cijepanja.

Brisanje: nadopuni prije ulaska — preraspodijeli ili spoji

- Ista disciplina kao pri umetanju, istim smjerom: jednom dolje, nikad natrag gore. Umetanje ne ulazi u pun čvor; brisanje ne ulazi u čvor koji je već na **minimumu** (t − 1 ključeva), jer bi uklanjanje ključa iz njega dovelo do **premale popunjenosti**. Zato se takav čvor najprije **nadopuni**.
- Prvo jeftini popravak: **preraspodjela** — susjed s viškom daje svoj najbliži ključ roditelju, a razdjelni ključ iz roditelja spušta se u tanki čvor. Redoslijed je fiksan: najprije lijevi susjed, zatim desni.
- Samo ako nijedan susjed nema višak: **spajanje** — spojite čvor sa susjedom (s lijevim ako postoji), povlačeći razdjelni ključ iz roditelja prema dolje.
- Ključ u unutarnjem čvoru ne smije se samo ukloniti (on razdvaja dvoje djece): zamijeni ga **prethodnik** (ili sljedbenik, ako lijevo dijete nema viška); ako su oba djeteta na minimumu, spoje se, a ključ koji se briše spušta se u sredinu spojenog čvora.
- Ako korijen ostane bez ključeva, odbacuje se i njegovo jedino dijete postaje novi korijen — jedini način na koji stablo gubi razinu.



Što B-stablo donosi te koliko košta

Donosi	Košta
O(log_t n) dohvata za svaku operaciju — podaci su udaljeni samo jedan dohvat bloka	Široki čvorovi, svaki u prosjeku otprilike trećinu prazan

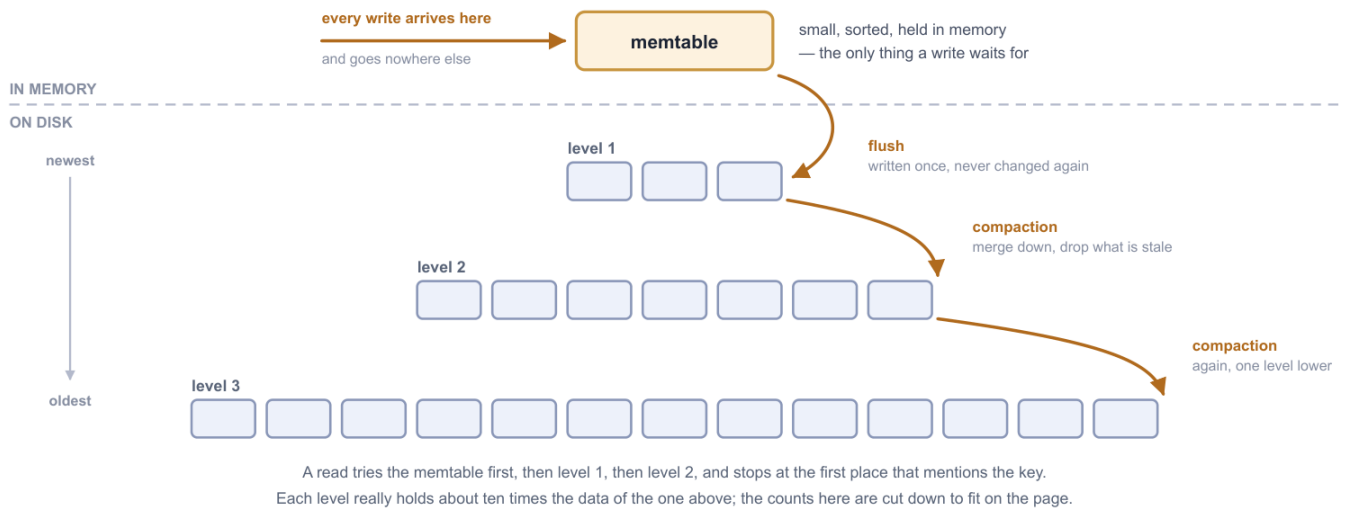
Odjeljak 6 — LSM-stablo

Motivacija: sada prevladavaju pisanja

- Sve su dosadašnje implementacije optimizirale *dohvat*. Ova mijenja ono što je skupo: **pisanja** pristižu brže nego što ih itko želi ponovno čitati.
- Nešto novo stiže svakih nekoliko sekundi, cijeli dan, i mora završiti ondje gdje se poslije može pronaći.
- Hod kojim se nešto sprema nikad nije bio *logički* nužan — pohrana ne mora znati gdje se nalazila stara inačica, nego samo da sada postoji novija.

Ideja: prestanite popravljati na mjestu, počnite slagati na hrpu

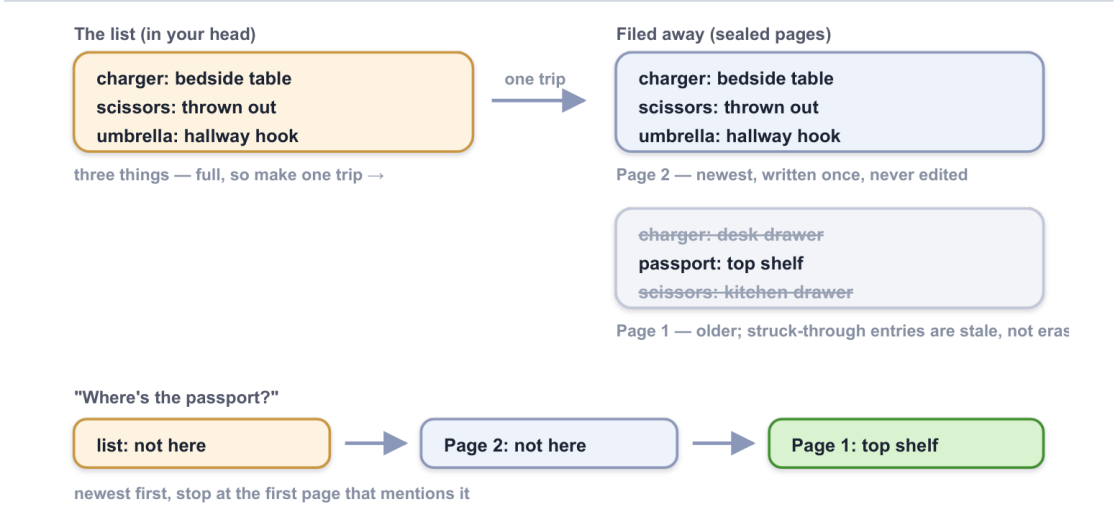
- Nova pisanja držite u malom, sortiranom međuspremniku u memoriji: **memorijskoj tablici (memtable)**.
- Kad se napuni, nemojte je spajati ni s čim — zapišite je izravno na disk, cijelu i sortiranu, te je više nikad ne dirajte.
- To je **stablo sa spajanjem strukturirano kao zapisnik (log-structured merge tree, LSM-stablo)**: *zapisnik* (pisanja se samo dodaju na kraj), *spajanje* (hrpa se usklađuje poslije, izvan kritičnog puta).



Razrađeni primjer: popis u vašoj glavi

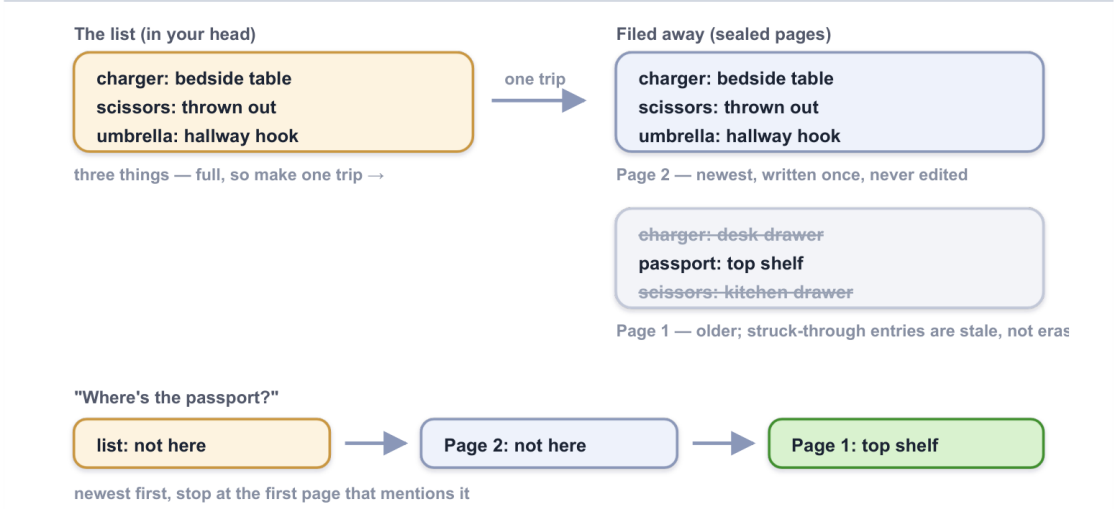
Vodite popis mjesta na kojima se stvari nalaze. Kad skupi tri stavke, odložite ga kao zapečaćenu stranicu i započnite novi popis.

- punjač → ladica radnog stola; putovnica → gornja polica; škare → kuhinjska ladica. Popis je pun → zapečaćen kao **Stranica 1**.
- punjač → noćni ormarić; škare → bačene; kišobran → vješalica u hodniku. Popis je pun → zapečaćen kao **Stranica 2**.
- Stranica 1 i dalje kaže da je punjač u ladici radnog stola. Nitko je ne ispravlja i nitko to nikad neće učiniti — Stranica 2 jednostavno je novija.



Čitanje: prvo najnovija stranica, stani na prvom pogotku

- "Gdje je punjač?": provjerite popis (ništa), zatim Stranicu 2 (pronađen — noćni ormarić). Zapis "ladica radnog stola" sa Stranice 1 nikad se ni ne provjerava.
- "Gdje su škare?": Stranica 2 kaže da ih nema (nadgrobna oznaka). Pretraga tu staje — zapis "kuhinjska ladica" ispod nje nikad se ne vidi, što je ispravno.
- "Gdje je pumpa za bicikl?": provjerena je svaka stranica, nije pronađena nigdje. Pitanje **bez odgovora skupo je** — to je jedini slučaj u kojem se mora provjeriti sve.



Datoteke na disku: SSTableovi i rijetki indeksi

- Svaka datoteka na disku (**SSTable**) ima sortiranu podatkovnu datoteku i mali indeks koji bilježi samo *prvi* ključ svakog bloka.
- Indeks može biti rijedak upravo zato što su podaci već sortirani: binarno pretražite mali indeks, a zatim pregledajte samo jedan blok na koji upućuje.
- Indeks temeljen na raspršenoj tablici to ne bi mogao — ne može vam reći gdje neki raspon *počinje*.

Ograničavanje hrpe: sažimanje i Bloomovi filtri

- **Sažimanje (compaction)**: povremeno spojite nekoliko datoteka u manji broj, usput odbacujući zastarjele zapise i potrošene nadgrobne oznake — tek se tada nadgrobna oznaka (i nevažeći zapis ispod nje) može konačno ukloniti.
- **Bloomov filtar**: mala, jeftina bitovna mapa ispred svake datoteke koja odgovara "sigurno nije ovdje" ili "možda je ovdje", s podesivom stopom lažno pozitivnih rezultata. Pri traženju odsutnog ključa preskače gotovo svaku datoteku.
- Gdje potrošiti memoriju na filtre zasebna je projektantska odluka koju različiti stvarni sustavi donose na različite načine.

Pretpostavka RUM: odaberite dvoje od troje

- **Pojačanje čitanja** — čitanje će možda morati provjeriti nekoliko datoteka.
- **Pojačanje pisanja** — podaci se ponovno zapisuju svaki put kad ih sažimanje pomakne na nižu razinu.
- **Memorijsko (prostorno) pojačanje** — zastarjeli zapisi i potrošene nadgrobne oznake zauzimaju prostor dok ih sažimanje ne dosegne.
- Svaka pristupna metoda može optimizirati dvije od tih veličina samo tako da popusti u trećoj.

Još jedan dio: zapisnik unaprijednog pisanja

- Memorijska tablica živi u memoriji — ne preživljava nestanak napajanja.
- Prije ulaska u memorijsku tablicu zapis se dodaje u **zapisnik unaprijednog pisanja (write-ahead log)**: jeftino (samo dodavanje na kraj) i osigurava da pad sustava ništa ne košta (ponovno izvedite zapisnik i potpuno obnovite memorijsku tablicu).
- Segment zapisnika odbacuje se tek kad se njegova memorijska tablica isprazni na disk — ne prije.

Što LSM-stablo donosi te koliko košta

Donosi	Košta
O(1) na kritičnom putu pisanja (samo dodavanje)	Čitanja možda provjeravaju nekoliko datoteka; sažimanje ponovno zapisuje podatke više puta

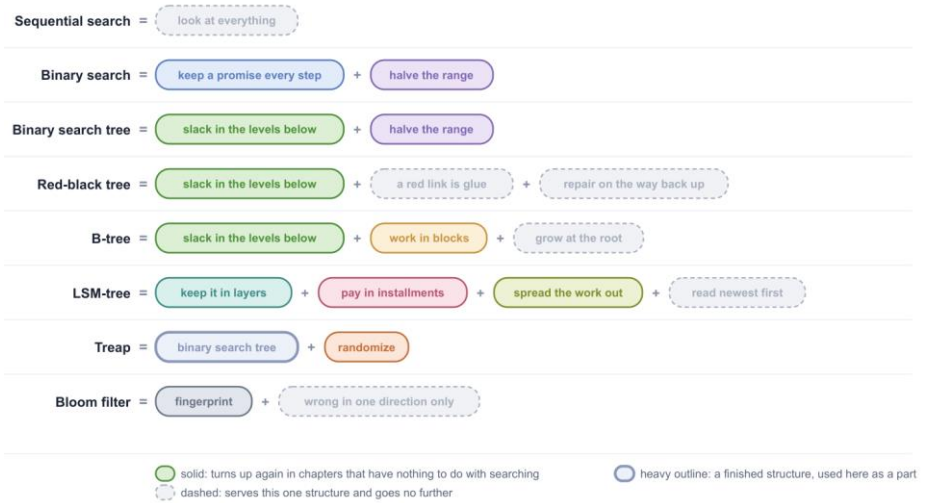
Odjeljak 7 — Sinteza i zaključak

Cjelovita usporedna tablica

Struktura	Pretraživanje	Umetanje	Čime se plaća ravnoteža	Najbolja kada
Sekvencijski pregled	Θ(n)	Θ(1) dodavanje na kraj, Θ(n) ako se prvo provjerava	ne plaća se ništa	skup je malen ili se upotrebljava jednom
Sortirano polje	Θ(lg n)	Θ(n)	ne plaća se ništa	statički podaci, samo za čitanje
Binarno stablo pretraživanja	Θ(h), h nije ograničen	Θ(h)	ne plaća se ništa	nasumičan redoslijed pristizanja i prihvatljiv vam je rizik
Crveno-crno stablo	Θ(lg n)	Θ(lg n)	popravak pri svakom umetanju	višenamjenska uređena mapa u memoriji
B-stablo	Θ(log_t n) dohvata	Θ(log_t n) dohvata	široki čvorovi, svaki otprilike trećinu prazan	podaci su udaljeni jedan dohvat bloka
LSM-stablo	nekoliko mjesta, prvo najnovije	Θ(1) na kritičnom putu	naknadno sažimanje; pojačanje pisanja	pisanja pristižu brže od čitanja

Istih šest struktura, devet ponovno upotrebljivih ideja

- **Održavajte obećanje u svakom koraku** — invarijanta binarnog pretraživanja, ponovno upotrijebljena posvuda. *Drugdje:* Dijkstrin skup obrađenih vrhova, sortiranje umetanjem.
- **Prepolovite raspon** — podijeli pa vladaj. *Drugdje:* sortiranje spajanjem, brza Fourierova transformacija.
- **Rezervni prostor u nižim razinama** — stablo uvijek ima mjesta ispod (BST, crveno-crno stablo i B-stablo). *Drugdje:* AVL stabla, preskočni popisi, gomile.
- **Radite u blokovima** — neka svaki skup odlazak donese više vrijednosti (B-stablo). *Drugdje:* diskovne stranice, linije priručne memorije.
- **Držite u slojevima** — mali brzi sloj u memoriji ispred velikih sporih na disku (LSM-stablo). *Drugdje:* preskočni popisi.
- **Plaćajte u obrocima (amortizacija)** — skupo spajanje naplatite mnogim pisanjima koja su ga izazvala (LSM-stablo). *Drugdje:* polje koje raste, union-find.
- **Rasporedite posao (deamortizacija)** — spajanje radite u malim komadima u pozadini, izvan puta pisanja, pa nijedno pisanje ne čeka na sve (LSM-stablo). *Drugdje:* raspršena tablica koja mijenja veličinu malo-pomalo, red u stvarnom vremenu.
- **Randomizirajte da izbjegnete najgori slučaj** — slučajni prioriteti (treap). *Drugdje:* pivot brzog sortiranja, raspršene tablice.
- **Otisak umjesto usporedbe** — Bloomov filtar. *Drugdje:* provjera podudaranja datoteka, Karp–Rabin.



Odabir pod pritiskom: navika odlučivanja, a ne dijagram toka

Prije nego što posegnete za strukturom, radnom opterećenju postavite tri pitanja:

1. Što je ovdje stvarno skupo — uspoređivanje, dohvaćanje ili pisanje?
2. Koliko se često stvari mijenjaju u odnosu na učestalost njihova traženja?
3. Trebaju li ikome ključevi ikada biti *uređeni* (upiti nad rasponom, najbliži susjed, rang)?

Jedna rečenica po strukturi (pomoć pri ponavljanju)

- Sekvencijsko pretraživanje: provjeri sve, ne obećavaj ništa.
- Sortirano polje: plati pri svakom umetanju da kupiš jeftino pretraživanje.
- Binarno stablo pretraživanja: jeftino ga je mijenjati, ali za visinu nitko nije odgovoran.
- Crveno-crno stablo: isto stablo, uz pravilo zbog kojeg netko postaje odgovoran za visinu.
- B-stablo: ponovno isto stablo, prilagođeno modelu troška u kojem dohvat nadmašuje uspoređivanje.
- LSM-stablo: prestani popravljati na mjestu; neka se zapisi gomilaju, a poslije ih uskladi.

Završna poruka

- Nijedna od ovih struktura nije "bolja" od drugih — svaka je pravi odgovor na drukčiju pretpostavku o radnom opterećenju.
- Korisna navika nije pamtiti šest struktura. Prije izbora treba pitati: što je ovdje jeftino, što skupo, koliko se često stvari mijenjaju i je li poredak važan?
- Odgovorite li iskreno, prava je struktura obično očita. Odgovorite li pogrešno (ili preskočite pitanje), možete izgraditi tehnički impresivno rješenje problema koji nitko nema.

Za sljedeći put / vježbe

- Vježbe 4, 17, 21 i 22 iz udžbenika najmanji su preporučeni skup ako nema dovoljno vremena.
- Vježbe 1–12 ("izvedi i opiši stanje", §2.10.1): praktične vježbe izgradnje/praćenja, dobre za problemski zadatak.
- Vježbe 13–19 ("koje je obećanje prekršeno", §2.10.2): stabla koja je "vratio model" — pronađite koje je od pet svojstava valjanog LLRB-a narušeno (jedno je stablo valjano).
- Vježbe 20–23 ("vi zadajete ulaz", §2.10.3): pronađite ulaz koji obara tvrdnju.
- Vježbe 24–32 ("za kojom idejom posežete", §2.12.4): devet pitanja za dijagnosticiranje radnog opterećenja, svaka od devet ideja točan je odgovor na točno jedno — dobra za brzu provjeru u učionici ili pitanja s glasanjem jer imaju višestruki izbor i naknadno obrazloženje.
- Vježba 33 ("testovi prolaze — što su propustili?", §2.12.5, ~20 min): kolegina LSM-pohrana prolazi sva tri testa, ali nakon nestanka napajanja nestane nekoliko stotina pisanja koja su pozivateljima već bila potvrđena — imenujte zahtjev koji testovi nikad nisu izrekli i opišite test koji bi pao (rješenje vodi do Slajda 54; ne otkrivajte ga unaprijed).
- Vježba 34 ("ispunite karticu", §2.12.6, ~40 min): sedam projektantskih pitanja za novo radno opterećenje (mreža meteoroloških postaja) — dobra kao domaća zadaća.
- Vježba 35 (dodatak, §2.15.3): najteža u poglavlju (~120 min).